

AI ASSISTED CODING

Lab Assignment-2.1

H.T.No:2303A510B1

B.SATHWIK REDDY

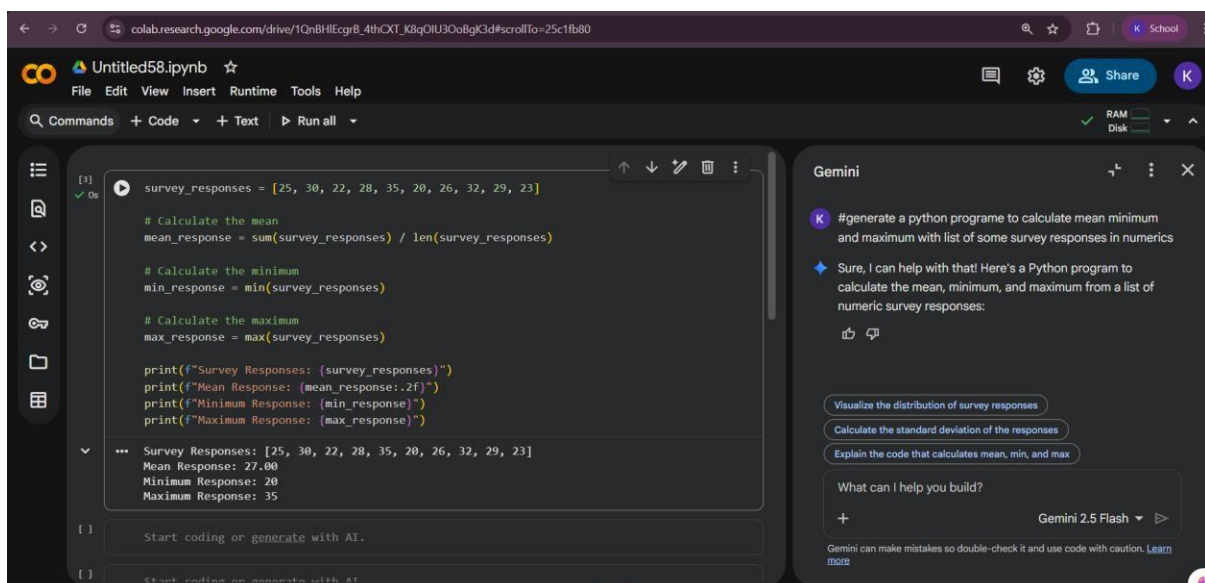
Batch:14

Task 1: Statistical Summary for Survey Data

#Prompt

generate a python programme to calculate mean minimum and maximum with list of some survey responses in numerics

Code:



```
[3] ✓ De
survey_responses = [25, 30, 22, 28, 35, 20, 26, 32, 29, 23]

# Calculate the mean
mean_response = sum(survey_responses) / len(survey_responses)

# Calculate the minimum
min_response = min(survey_responses)

# Calculate the maximum
max_response = max(survey_responses)

print(f"Survey Responses: {survey_responses}")
print(f"Mean Response: {mean_response:.2f}")
print(f"Minimum Response: {min_response}")
print(f"Maximum Response: {max_response}")

... Survey Responses: [25, 30, 22, 28, 35, 20, 26, 32, 29, 23]
Mean Response: 27.00
Minimum Response: 20
Maximum Response: 35
```

Start coding or generate with AI.

Start coding or generate with AI.

Gemini

#generate a python programme to calculate mean minimum and maximum with list of some survey responses in numerics

Sure, I can help with that! Here's a Python program to calculate the mean, minimum, and maximum from a list of numeric survey responses:

Visualize the distribution of survey responses

Calculate the standard deviation of the responses

Explain the code that calculates mean, min, and max

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Output:

```
... Survey Responses: [25, 30, 22, 28, 35, 20, 26, 32, 29, 23]  
    Mean Response: 27.00  
    Minimum Response: 20  
    Maximum Response: 35
```

Justification:

Google Gemini efficiently generated a correct Python function to compute the mean, minimum, and maximum from survey data.

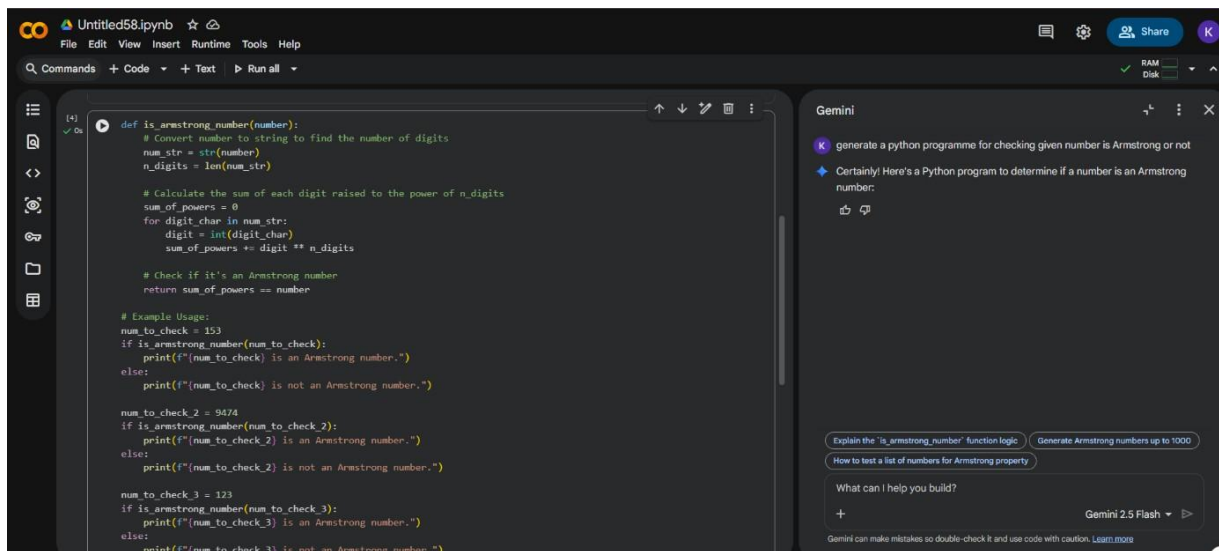
The output displayed in Colab verified the accuracy of the generated logic with sample inputs.

Task-2: Armstrong Number - AI Comparison

#Prompt:

Using google colab

Code:



Output:

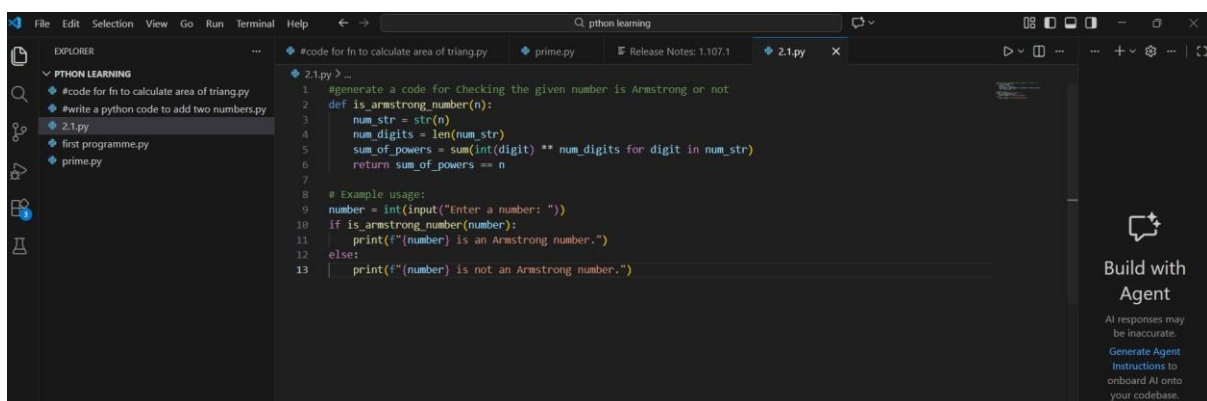
```

*** 153 is an Armstrong number.
    9474 is an Armstrong number.
    123 is not an Armstrong number.

```

Using Github copilot

Code:



Output:



```
"c:/Users/nithe/OneDrive/Documents/pthon_learning/2.1.py"
Enter a number: 234
234 is not an Armstrong number.
PS C:\Users\nithe\OneDrive\Documents\pthon_learning>
PS C:\Users\nithe\OneDrive\Documents\pthon_learning> & C:/Users/nithe/AppData/Local/Python/pythoncore-3.14-64/python.exe
"c:/Users/nithe/OneDrive/Documents/pthon_learning/2.1.py"
Enter a number: 153
153 is an Armstrong number.
PS C:\Users\nithe\OneDrive\Documents\pthon_learning>
```

Justification:

Gemini generated a clear, beginner-friendly Armstrong number solution with step-by-step logic and explanation.

GitHub Copilot produced a more concise and optimized implementation but with minimal explanation.

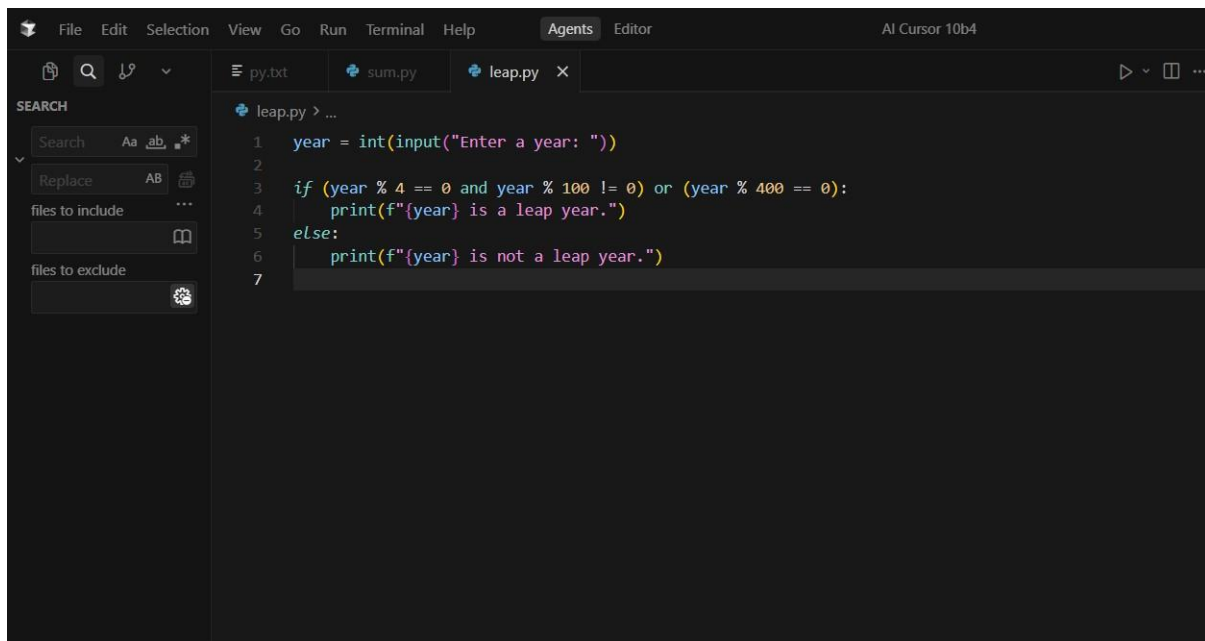
This comparison shows that Gemini prioritizes clarity and learning, while Copilot focuses on efficiency and developer productivity.

Task-3: Leap Year Validation Using Cursor AI

#Prompt

Generate a python code for checking weather the given year is leap year or not

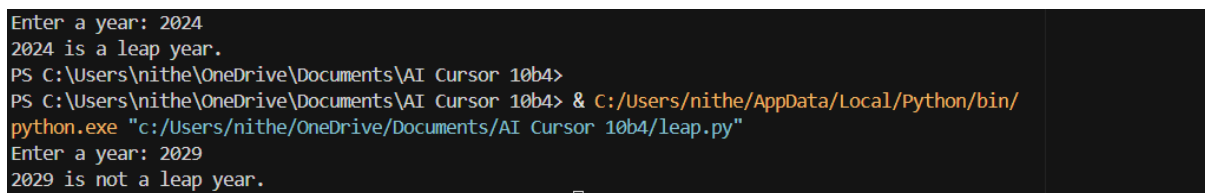
Code:



The screenshot shows a code editor with a dark theme. The menu bar includes File, Edit, Selection, View, Go, Run, Terminal, Help, Agents, and Editor. The status bar at the top right says 'AI Cursor 10b4'. The file explorer on the left shows a search bar and a list of files: py.txt, sum.py, and leap.py. The main editor area displays the following Python code in leap.py:

```
1 year = int(input("Enter a year: "))
2
3 if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
4     print(f"{year} is a leap year.")
5 else:
6     print(f"{year} is not a leap year.")
7
```

Output:



The screenshot shows a terminal window with the following output:

```
Enter a year: 2024
2024 is a leap year.
PS C:\Users\nithe\OneDrive\Documents\AI Cursor 10b4>
PS C:\Users\nithe\OneDrive\Documents\AI Cursor 10b4> & C:/Users/nithe/AppData/Local/Python/bin/
python.exe "c:/Users/nithe/OneDrive/Documents/AI Cursor 10b4/leap.py"
Enter a year: 2029
2029 is not a leap year.
```

Justification:

Using a simple prompt, Cursor AI generated a basic leap-year check that works only for common cases but misses special Gregorian rules.

A more detailed prompt led to a correct, reusable solution that follows all leap-year conditions.

Task-4: Student Logic + AI Refactoring (Odd/Even Sum)

Without AI

#Prompt

Write a Python program manually to calculate the sum of odd numbers and the sum of even numbers from a given tuple using basic control structures

Code:

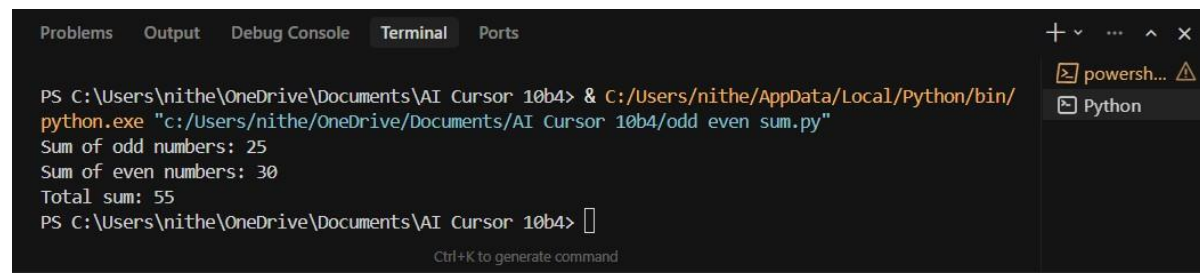
```
numbers = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10) # example tuple

sum_odd = 0
sum_even = 0

for number in numbers:
    if number % 2 == 0:
        sum_even += number
    else:
        sum_odd += number

print(f"Sum of odd numbers: {sum_odd}")
print(f"Sum of even numbers: {sum_even}")
print(f"Total sum: {sum_odd + sum_even}")
```

Output:



The screenshot shows a terminal window with the following content:

```
PS C:\Users\nithe\OneDrive\Documents\AI Cursor 10b4> & C:/Users/nithe/AppData/Local/Python/bin/python.exe "c:/Users/nithe/OneDrive/Documents/AI Cursor 10b4/odd even sum.py"
Sum of odd numbers: 25
Sum of even numbers: 30
Total sum: 55
PS C:\Users\nithe\OneDrive\Documents\AI Cursor 10b4> 
```

The terminal window has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. On the right side, there are icons for 'powershell' and 'Python'.

AI Refactoring:

#Prompt

Refactor this Python code to make it more readable, reusable, and Pythonic without changing the output

Code:

```
# Refactored to increase readability and Pythonic style.  
def print_odd_even_sum(numbers):  
    sum_odd, sum_even, total = sum_odd_even(numbers)  
    print(f"Sum of odd numbers: {sum_odd}")  
    print(f"Sum of even numbers: {sum_even}")  
    print(f"Total sum: {total}")  
  
# example usage  
print_odd_even_sum(numbers)  
# Refactored to increase readability and Pythonic style.
```

Output:

```
Sum of odd numbers: 25  
Sum of even numbers: 30  
Total sum: 55  
PS C:\Users\nithe\OneDrive\Documents\AI Cursor 10b4> █
```

Ctrl+K to generate command

Justification:

Writing the code manually first ensures a clear understanding of the problem and the underlying logic.

It helps demonstrate individual problem-solving skills without relying on AI assistance.

